

Day 12 : SQLite Local Database

1. Introduction to SQLite

SQLite is a lightweight local database used to store data permanently on the user's device. It is useful for apps like Notes, Todo Lists, Expense Trackers, and Offline Apps where data must remain even after closing the app.

2. Why Do We Use it?

- Store data locally on device
- Works offline
- Fast data access
- Permanent data storage
- Supports CRUD operations
- No internet required

3. Required Packages

In pubspec.yaml :

```
dependencies:  
  flutter:  
    sdk: flutter  
  sqflite: ^2.3.0  
  path: ^1.8.3
```

4.DatabaseHelper Class

```
import 'package:sqflite/sqflite.dart';  
import 'package:path/path.dart';  
  
class DatabaseHelper {  
  static Database? _database;  
  
  Future<Database> get database async {  
    if (_database != null) {  
      return _database!  
    }  
    _database = await initDatabase();  
    return _database!  
  }  
  
  Future<Database> initDatabase() async {  
    String path = join(await getDatabasesPath(), 'notes.db');  
  
    return await openDatabase(  
      path,  
      version: 1,  
      onCreate: (db, version) async {  
        await db.execute(  
          'CREATE TABLE notes(id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT,  
          );  
        },  
      );  
    }  
  }  
}
```

5. Data Model

Note Model :

```
class Note {
  int? id;
  String title;
  String content;

  Note({
    this.id,
    required this.title,
    required this.content,
  });

  Map<String, dynamic> toMap() {
    return {
      'id': id,
      'title': title,
      'content': content,
    };
  }

  factory Note.fromMap(Map<String, dynamic> map) {
    return Note(
      id: map['id'],
      title: map['title'],
      content: map['content'],
    );
  }
}
```

6. CRUD Operation

1. Insert Data

```
Future<int> insertNote(Note note) async {
  final db = await database;
  return await db.insert('notes', note.toMap());
}
```

Usage:

```
await insertNote(
  Note(
    title: "Flutter",
    content: "Learning SQLite",
  ),
);
```

2. Read all data

```
Future<List<Map<String, dynamic>>> getNotes() async {  
  final db = await database;  
  return await db.query('notes');  
}
```

3. Read Single Record

```
Future<List<Map<String, dynamic>>> getNote(int id) async {  
  final db = await database;  
  
  return await db.query(  
    'notes',  
    where: 'id = ?',  
    whereArgs: [id],  
  );  
}
```

4. Update data

```
Future<int> updateNote(Note note) async {  
  final db = await database;  
  
  return await db.update(  
    'notes',  
    note.toMap(),  
    where: 'id = ?',  
    whereArgs: [note.id],  
  );  
}
```

5. Delete Data

```
Future<int> deleteNote(int id) async {  
  final db = await database;  
  
  return await db.delete(  
    'notes',  
    where: 'id = ?',  
    whereArgs: [id],  
  );  
}
```

7 . SQL Commands Used

Create Table

```
CREATE TABLE notes(  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
title TEXT,  
content TEXT  
)
```

Insert

```
INSERT INTO notes(title,content)  
VALUES('Flutter','SQLite')
```

Select

```
SELECT * FROM notes
```

Update

```
UPDATE notes  
SET title='Updated'  
WHERE id=1
```

Delete

```
DELETE FROM notes  
WHERE id=1
```

8. Query Example

1. WHERE Clause

```
db.query(  
  'notes',  
  where: 'title = ?',  
  whereArgs: ['Flutter'],  
);
```

2. Order By

```
db.query(  
  'notes',  
  orderBy: 'id DESC',  
);
```

3. Limit

```
db.query(  
  'notes',  
  limit: 5,  
);
```

Code :

Note.dart

```
class Note {  
  int? id;  
  String title;  
  String content;  
  
  Note({  
    this.id,  
    required this.title,  
    required this.content,  
  });  
  
  Map<String, dynamic> toMap() {  
    return {  
      'id': id,  
      'title': title,  
      'content': content,  
    };  
  }  
  
  factory Note.fromMap(Map<String, dynamic> map) {  
    return Note(  
      id: map['id'],  
      title: map['title'],  
      content: map['content'],  
    );  
  }  
}
```

Databasehelper.dart

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
import 'note.dart';

class DatabaseHelper {
  static Database? _database;

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await initDB();
    return _database!;
  }

  Future<Database> initDB() async {
    String path = join(await getDatabasesPath(), 'notes.db');

    return await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE notes(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT,
            content TEXT
          )
        ''');
      },
    );
  }

  Future<int> insertNote(Note note) async {
    final db = await database;
    return await db.insert('notes', note.toMap());
  }
}
```

```
Future<int> insertNote(Note note) async {
  final db = await database;
  return await db.insert('notes', note.toMap());
}

Future<List<Note>> getNotes() async {
  final db = await database;

  List<Map<String, dynamic>> maps =
    await db.query('notes', orderBy: 'id DESC');

  return List.generate(
    maps.length,
    (i) => Note.fromMap(maps[i]),
  );
}

Future<int> updateNote(Note note) async {
  final db = await database;

  return await db.update(
    'notes',
    note.toMap(),
    where: 'id=?',
    whereArgs: [note.id],
  );
}

Future<int> deleteNote(int id) async {
  final db = await database;

  return await db.delete(
    'notes',
    where: 'id=?',
    whereArgs: [id],
  );
}
}
```

Main.dart

```
import 'package:flutter/material.dart';
import 'database_helper.dart';
import 'note.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: NotesPage(),
    );
  }
}

class NotesPage extends StatefulWidget {
  @override
  State<NotesPage> createState() => _NotesPageState();
}

class _NotesPageState extends State<NotesPage> {
  DatabaseHelper dbHelper = DatabaseHelper();

  List<Note> notes = [];

  TextEditingController titleController =
    TextEditingController();

  TextEditingController contentController =
    TextEditingController();

  @override
  void initState() {
    super.initState();
    loadNotes();
  }
}
```

```
Future<void> loadNotes() async {
  notes = await dbHelper.getNotes();
  setState(() {});
}

Future<void> addNote() async {
  await dbHelper.insertNote(
    Note(
      title: titleController.text,
      content: contentController.text,
    ),
  );

  titleController.clear();
  contentController.clear();

  loadNotes();
}

Future<void> deleteNote(int id) async {
  await dbHelper.deleteNote(id);
  loadNotes();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text("SQLite Notes App"),
    ),

    body: Column(
      children: [
        TextField(
          controller: titleController,
          decoration:
            InputDecoration(labelText: "Title"),
        ),
      ],
    ),
  );
}
```

```
        ),  
        TextField(  
            controller: contentController,  
            decoration:  
                InputDecoration(labelText: "Content"),  
        ),  
        ElevatedButton(  
            onPressed: addNote,  
            child: Text("Add Note"),  
        ),  
        Expanded(  
            child: ListView.builder(  
                itemCount: notes.length,  
                itemBuilder: (context, index) {  
                    return ListTile(  
                        title: Text(notes[index].title),  
                        subtitle: Text(notes[index].content),  
                        trailing: IconButton(  
                            icon: Icon(Icons.delete),  
                            onPressed: () {  
                                deleteNote(notes[index].id!);  
                            },  
                        ),  
                    );  
                },  
            );  
        ],  
    ),  
],  
),
```

OUTPUT :

